Chapter 19

Reinforcement Learning in Finance

19.1 Introduction

Every model built so far in this book, however sophisticated, has answered a single question once. Chapter 6's classifiers predict a label from a feature vector; Chapter 9's credit models predict a default probability; Chapter 13's Almgren-Chriss trajectory prescribes an entire trading schedule, but it does so by solving one optimization problem in advance and then executing the resulting plan. None of these approaches change their answer in response to how the world responds to the decisions already made.

A portfolio manager, a market maker, and a trading desk do not operate this way: they observe an outcome, update their beliefs, and choose their next action accordingly, over and over, for as long as the position stays open. Reinforcement learning is the branch of machine learning built for this setting, sequential decision-making under uncertainty, where an agent's own actions influence the future states it will face, and where the right action today depends on the consequences it sets up for tomorrow, not only on today's immediate reward.

This chapter formalizes that setting, the Markov decision process, and develops the two families of algorithms built to solve it: value-based methods, culminating in Q-learning and its neural-network extension, and policy-based methods, culminating in the policy-gradient algorithms that power most modern applications. It builds on machinery this book has already developed: Chapter 5's binomial-tree early-exercise decision, Chapter 12's portfolio rebalancing problem, and Chapter 13's optimal-execution and market-making models are all special cases of the general sequential-decision problem, simple enough that a closed-form or backward-induction solution exists. Reinforcement learning is what a practitioner reaches for when the problem is still sequential but no longer simple enough for a closed form, a common situation once market impact, transaction costs, and a client's actual risk preferences no longer fit the clean functional forms those chapters assumed.

The chapter's central worked example makes this connection concrete. It reframes Chapter 5's American put early-exercise decision as a three-state Markov decision process, solves it with model-free Q-learning instead of backward induction, and shows that the learned answer converges to the value the tree already established: a small, verifiable proof that reinforcement learning recovers a known answer, before the chapter asks the reader to trust it on problems with no closed form.

The chapter proceeds in three stages. Sections 19.2.1 through 19.2.6 build the Markov decision process framework, the Bellman equation, and the vocabulary (model-based versus model-free, on-policy versus off-policy) needed to place every algorithm that follows. Sections 19.3.1 through 19.3.3 develop the algorithms, from the verifiable tabular Q-learning example to deep Q-networks and policy-gradient methods for problems too large for a table. The remaining sections turn to application and assessment: optimal execution, market making, and portfolio management (Sections 19.4.1 through 19.4.6), an industry case study (Section 19.5.1), and the evaluation methodology and risks (Sections 19.5.2 and 19.6) that separate a system that merely impresses in a backtest from one that is safe to deploy with real capital.

Wulin.Suo@Queensu.ca

19.2 The Reinforcement Learning Framework

This part sets up the sequential decision problem and the value functions that make it tractable.

19.2.1 Markov Decision Processes: States, Actions, Rewards, and Policies

Consider a market maker deciding, moment by moment, how to skew its bid and ask quotes as its inventory evolves, or a trader deciding how many shares of a large order to release this minute versus holding back for a better price later. In both cases today's decision changes tomorrow's starting position, so the decisions cannot be solved one at a time, as most models in Chapters 12 and 13 solve them. A reinforcement learning problem is formalized as a Markov decision process, a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, to capture this kind of interlocking, path-dependent problem.

At each discrete time step t, an **agent** observes a **state** $s_t \in \mathcal{S}$, a summary of everything about the environment relevant to the decision (a market maker's current inventory and the prevailing bid-ask spread; a trading agent's remaining order size and elapsed time; a portfolio manager's current holdings and estimated risk factors). The agent selects an **action** $a_t \in \mathcal{A}$ (post a quote at a given skew; sell a given number of shares this period; rebalance toward a given target weight) according to a **policy** $\pi(a \mid s)$, the probability of taking action a in state s, which maps each state to a distribution over the available actions with $\sum_a \pi(a \mid s) = 1$. A policy that puts all its probability on one action per state is deterministic: the agent always does the same thing from the same state.

The environment then transitions to a new state s_{t+1} according to a transition function $P(s_{t+1} \mid s_t, a_t)$ and returns a scalar **reward** r_{t+1} (realized profit and loss net of costs; a small quadratic risk penalty; an exercise payoff), governed by a reward function $R(s_t, a_t, s_{t+1})$. Figure 19.1 depicts this loop; it repeats until the episode ends, either at a fixed horizon (the market closes, the order is fully executed) or when the agent reaches an absorbing state (a position is liquidated, an option is exercised).

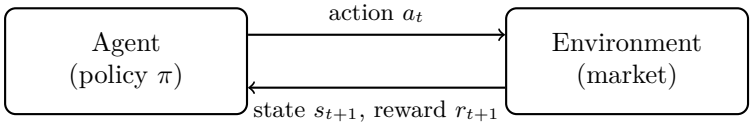


Figure 19.1: The reinforcement learning loop

The word "Markov" is a precise mathematical assumption: $P(s_{t+1} \mid s_t, a_t)$ and $R(s_t, a_t, s_{t+1})$ depend only on the current state and action, not on the history that led there. Every stochastic process this book has used to model prices makes the same assumption, from the binomial tree's independent up-and-down moves in Chapter 5 to the random-walk and GARCH innovations of Chapter 7; it is a familiar assumption under a new name, now applied to an agent's decisions as well as to prices. Whether a proposed state representation is Markovian is itself a design choice: a market maker's state that includes only the current mid-price, and not recent order flow, discards information a real market maker would use, which is why Chapter 14's order-flow-imbalance regression and Chapter 13's Kyle's-lambda framework are natural sources of state features for a trading agent, not only standalone predictive signals.

The agent's goal is to choose a policy that maximizes the expected **return**, the discounted sum of future rewards,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \tag{19.1}$$

where the **discount factor** $\gamma \in [0, 1]$ plays the role of the discount factor $1/(1+r)$ in Chapter 5's valuation formulas: a reward received later is worth less today, both because of the time value of money and because a distant reward is a noisier, harder-to-credit consequence of the current action. A myopic agent with $\gamma = 0$ maximizes only the immediate reward, the one-shot optimization earlier chapters' models perform; the closer γ is to 1, the more the current choice is shaped by consequences many steps away, the structure a sequential problem like optimal execution or ongoing portfolio management requires and a one-shot model cannot represent.

Wulin.Suo@Queensu.ca

19.2.2 From One-Shot Optimization to Sequential Decisions

Why the models of Chapters 12 and 13 did not need this machinery clarifies what reinforcement learning adds. The Almgren-Chriss optimal-execution model (Chapter 13's Section 13.3.5) assumes a known functional form for market impact, quadratic in the trade rate, and a known risk-aversion parameter ϕ; given those, the optimal trajectory follows in closed form in one step, with nothing to learn from experience, because the model already specifies how the environment responds to every action. Chapter 13's Avellaneda-Stoikov inventory model (Section 13.4) is more dynamic, recomputing the market maker's quotes at every instant as inventory and time-to-close evolve, but its update rule is a closed-form formula derived from an assumed order-arrival process, not a policy learned by trial and error. Chapter 12's Section 12.4.1 allocation discussion goes furthest toward the sequential nature of real portfolio management, noting that rebalancing recurs and that transaction costs make each decision depend on the current position, but it treats each rebalancing as a separate, myopic optimization. Each model is right within its assumptions and remains the correct tool whenever they hold: a well-estimated impact function and a well-specified risk-aversion parameter make a closed-form solution more accurate and far cheaper to compute than a learned one.

Reinforcement learning becomes the better tool when that closed form is unavailable: when the true market-impact function fits no simple parametric form, when transaction costs interact with the portfolio's history in ways too complex to solve analytically, or when the reward the agent cares about, a client's realized utility net of taxes, fees, and tolerance for drawdowns, does not reduce to a quadratic cost. In each case the problem is still a sequential decision problem as formalized in Section 19.2.1; what changes is that no one can write down P and R in closed form, which rules out the direct optimization of Chapters 12 and 13 and motivates learning a policy from simulated or historical experience.

19.2.3 The Simplest Case: Multi-Armed Bandits

A trading desk sending orders to market faces a choice most descriptions of execution skip: which venue to send each order to. A large-cap U.S. equity trades on a dozen or more venues, and the same order routed to two of them comes back at different prices, because each venue has its own resting liquidity, fee schedule, and population of counterparties. A desk that knew each venue's average price improvement would route everything to the best one. No desk knows this, and the only way to find out what a venue delivers is to send it orders, so the desk pays, in worse fills, for every order spent learning rather than earning.

This tension separates reinforcement learning from Chapters 6 and 7. A supervised model is handed a fixed dataset and never influences what it sees next; here the agent's own choices determine which data it observes, so an action that looks bad early may never be tried again and its true value never discovered. The problem in its purest form is the **multi-armed bandit**, named for a row of slot machines with unknown payout rates: at each step the agent picks one of K actions, called **arms**, and receives a noisy reward drawn from that arm's fixed but unknown distribution.

The bandit is the right place to start because of what it leaves out. There is one state, which never changes; the reward depends only on the action just taken; and no action moves the agent anywhere. Section 19.2.5's Bellman equation, with its discounting, bootstrapping, and credit assignment across a trajectory, is unnecessary, because $Q(a)$ is the mean reward of arm a, estimable by averaging. The one difficulty left is the one this chapter meets everywhere: the **exploration-exploitation trade-off**, choosing between the arm that currently looks best and one whose value is still uncertain.

19.2.4 Worked Example 19.1: Routing Orders Across Three Venues

A desk routes each arriving order to one of three venues. Measured in basis points of price improvement against the arrival mid-price, venue A delivers a mean of 1.2, venue B a mean of 2.0, and venue C a mean of 0.5, each with a standard deviation of 2.0 basis points. Those means are stated here so the example can be checked; the desk does not know them, and must infer them from the fills it receives. Since the standard deviation of a single fill is 2.5 times the 0.8 basis-point gap between the best venue and the second best, one order tells the desk almost nothing.

Wulin.Suo@Queensu.ca

The desk keeps an estimate $Q(a)$ of each venue's mean and a count $N(a)$ of orders sent there, updating after each fill by the running-average rule

$$Q(a) \leftarrow Q(a) + \frac{1}{N(a)}\big[r - Q(a)\big], \tag{19.2}$$

which is the sample mean of the rewards seen from a, written so it can be updated one fill at a time without storing them. After sending one order to each venue to get started, it follows an ε-greedy policy with $\varepsilon = 0.2$: with probability 0.2 route to a uniformly random venue, otherwise route to whichever venue currently has the highest Q. Table 19.1 traces the first twelve orders.

Order	Rule	Venue	Fill	$Q(A)$	$Q(B)$	$Q(C)$
1	warm-up	A	1.45	1.450	—	—
2	warm-up	B	1.74	1.450	1.740	—
3	warm-up	C	1.78	1.450	1.740	1.780
4	explore	A	1.92	1.685	1.740	1.780
5	exploit	C	2.39	1.685	1.740	2.085
6	exploit	C	−2.03	1.685	1.740	0.713
7	exploit	B	2.08	1.685	1.910	0.713
8	exploit	B	1.56	1.685	1.793	0.713
9	exploit	B	0.54	1.685	1.480	0.713
10	exploit	A	0.57	1.313	1.480	0.713
11	exploit	B	4.09	1.313	2.002	0.713
12	explore	C	3.23	1.313	2.002	1.342

Table 19.1: The first twelve orders of the venue-routing bandit. Fills and estimates are in basis points of price improvement; a dash marks a venue not yet tried. Every estimate follows from the running-average rule applied to the fills above it, so each row can be checked by hand.

The opening rounds show the failure mode that makes exploration necessary. The single warm-up order to venue C returned 1.78 basis points, a good fill drawn by chance from the *worst* of the three venues, and that one lucky draw made C the greedy choice. Orders 5 and 6 went there. What rescued the desk was not a clever exploration rule but repetition: order 6 came back at -2.03, dragging $Q(C)$ from 2.085 down to 0.713 and demoting C below both rivals. Exploiting a wrong estimate long enough tends to correct it, because every exploit is also a sample. By order 12 the desk has $Q(B)$ highest and is routing to the best venue, though $Q(A) = 1.313$ and $Q(C) = 1.342$ remain badly estimated from three and four orders respectively.

Run the same policy for 5,000 orders and the estimates sharpen to $Q(A) = 1.25$, $Q(B) = 1.99$, and $Q(C) = 0.40$, against true means of 1.2, 2.0, and 0.5. The desk routes 86.2% of its orders to B and realizes an average of 1.826 basis points per order.

That average is visibly short of the 2.0 basis points a desk that knew the answer would earn. The shortfall, 868 basis points in total across the 5,000 orders, is the **regret**: what learning cost, measured against an oracle that never had to learn. Most of it comes not from the early mistakes, which are cheap and brief, but from the ε-greedy rule itself, which keeps routing one order in five at random long after the answer is settled. That permanent tax is

$$\varepsilon \times \big(2.0 - \tfrac{1}{3}(1.2 + 2.0 + 0.5)\big) = 0.2 \times 0.7667 = 0.1533 \text{ basis points per order},$$

which is why the asymptotic average is $0.8 \times 2.0 + 0.2 \times 1.2333 = 1.8467$ rather than 2.0, and why the realized 1.826 sits just below it. The remedy is to explore hard early and taper off: decaying ε over the same 5,000 orders lifts the share routed to B from 86.2% to 95.2% and the average fill from 1.826 to 1.943 basis points, while keeping the early exploration that found B. Every later worked example in this chapter uses a decayed exploration schedule for this reason.

Wulin.Suo@Queensu.ca

Exploring Where It Pays: Upper Confidence Bounds and Thompson Sampling

Decaying ε fixes *how often* the desk explores but not *where*: an exploratory order is still as likely to go to venue C, which the data condemned long ago, as to venue A, which might yet be competitive. Two better rules direct exploration toward the venues whose value is still uncertain. The **upper confidence bound** (UCB) rule of Auer et al. (2002) routes each order to the venue with the largest optimistic estimate,

$$a_t = \arg\max_a \left[Q(a) + \sigma \sqrt{\frac{2 \ln t}{N(a)}} \right], \tag{19.3}$$

where $\sigma = 2.0$ basis points is the fill noise and t the order number. The bonus is large for a venue sampled rarely and shrinks as $N(a)$ grows, so a venue is revisited only for as long as its plausible upside could still beat the leader; the slowly growing $\ln t$ guarantees that no venue is abandoned forever on the strength of a few unlucky fills. **Thompson sampling** takes a Bayesian route to the same end (Russo et al. 2018). Under a flat prior and the known noise σ, the posterior for venue a's mean is normal with mean $Q(a)$ and standard deviation $\sigma/\sqrt{N(a)}$; before each order the desk draws one plausible mean from each venue's posterior and routes to the largest draw. A venue therefore receives each order with exactly the probability that it is the best venue given the fills so far, so exploration fades on its own as the posteriors tighten.

Run on the same three venues, with the same warm-up and the same seed as the ε-greedy desk, the two rules route 96.7% (UCB) and 98.5% (Thompson sampling) of 5,000 orders to venue B, against 86.2% for fixed and 95.2% for decayed ε, and earn average fills of 1.961 and 1.971 basis points against 1.826 and 1.943. Realized regret falls from 868 basis points to 286 (decayed ε), 195 (UCB) and 144 (Thompson sampling). A single run mixes the policy's choices with the luck of the fills, so Figure 19.2 averages the *expected* regret, the sum over orders of the gap between venue B's mean and the chosen venue's mean, across 200 seeds. The fixed-ε curve is a straight line: over the last 1,000 orders it adds 0.152 basis points per order, the permanent 0.1533 tax computed above. Decayed ε bends but is still adding 0.028 per order at the end, while UCB and Thompson sampling add only 0.005 and 0.003, finishing at 103 and 74 basis points of cumulative regret against 794 for fixed ε. Their curves flatten in the way theory predicts: Lai and Robbins (1985) showed that no rule can keep regret growing more slowly than the logarithm of the number of rounds, and UCB and Thompson sampling both achieve that logarithmic rate (Lattimore and Szepesvári 2020).

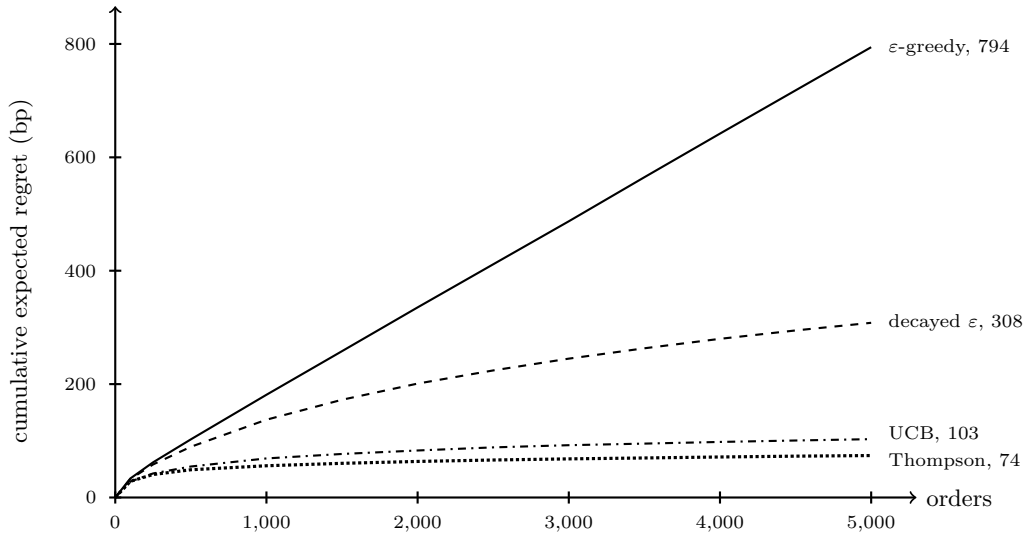


Figure 19.2: Cumulative expected regret of four venue-routing rules, averaged over 200 seeds of 5,000 orders each. Fixed ε (solid) grows linearly; decayed ε (dashed) bends; UCB (dash-dot) and Thompson sampling (dotted) flatten toward logarithmic growth.

Wulin.Suo@Queensu.ca

When the Best Venue Depends on the Order: Contextual Bandits

The three-venue desk assumed that one venue is best for every order. In practice the answer depends on what the desk observes when the order arrives: a small order may do best on a venue with tight quotes but little depth, a large one on a venue whose hidden liquidity absorbs size, and the ranking can shift with the spread, the time of day, or recent volatility. A **contextual bandit** lets the reward depend on a feature vector x_t observed before acting and learns a value $Q(a, x)$ for each action as a function of context, typically a regression per arm such as $Q(a, x) = \theta_a^\top x$. The exploration ideas carry over: LinUCB, introduced by Li et al. (2010) for news-article recommendation, adds to each arm's regression prediction a bonus proportional to that prediction's standard error, which is equation (19.3) with the regression's uncertainty in place of $\sigma/\sqrt{N(a)}$. A contextual bandit is still not a full reinforcement learning problem, since the context arrives from outside and today's routing decision does not change tomorrow's context. That makes it attractive for routing, pricing, and recommendation decisions that recur thousands of times a day, and it is also its limit: once an action changes the next state, as when a large order's impact moves the price the rest of the order will face, the problem needs the machinery of the next subsection.

Two threads run forward from here. Add a next state, so that today's action changes tomorrow's situation, and averaging is no longer enough: the agent must reason about consequences that arrive later, which Section 19.2.5's Bellman equation supplies and Section 19.3.1's Q-learning example puts to work. Keep the single state but shrink the gap between arms relative to the noise, and even this simplest problem becomes hard: Section 19.4.4 returns to a bandit whose best and second-best arms differ by so little against so much noise that 200,000 rounds still leave the agent on the wrong one.

19.2.5 Value Functions and the Bellman Equation

Suppose two agents follow the same policy but start from different states, one with a small, comfortable inventory and one deep in an overextended position: the policy alone cannot say how much trouble each is in, only what each plans to do next. Two functions answer this by summarizing how good a state, or state-action pair, is under a policy π: the **state-value function** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$, the expected return starting from state s and following π thereafter, and the **action-value function** $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$, the expected return from taking action a in state s and following π afterward.

Because the return decomposes into an immediate reward plus a discounted continuation, both functions satisfy a recursive relationship, the **Bellman equation**, named for Richard Bellman's original work on dynamic programming:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \Big[R(s, a, s') + \gamma V^\pi(s') \Big]. \tag{19.4}$$

The **Bellman optimality equation** specializes this to the best possible policy, replacing the expectation over actions with a maximization:

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) \Big[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \Big]. \tag{19.5}$$

A reader who worked through Chapter 5's two-step binomial tree (Section 5.4.2) has already solved a Bellman optimality equation by hand under another name. At each node, backward induction compares the value of exercising immediately with the discounted expected value of continuing, and takes the larger: the $\max_{a'}$ above, applied to the actions "exercise" and "hold." Section 19.3.1 makes the correspondence exact by reformulating that tree as a Markov decision process and re-deriving the same option value. Dynamic programming methods, value iteration and policy iteration, solve the Bellman equation directly whenever P and R are fully known, as Chapter 5's tree does. Reinforcement learning's model-free methods, developed next, estimate Q^* from sampled transitions and rewards alone, which makes them applicable where, as in most real trading and portfolio problems, no one can specify P and R in closed form.

Wulin.Suo@Queensu.ca

19.2.6 A Taxonomy of Reinforcement Learning Methods

Two further distinctions organize the many reinforcement learning algorithms in use, and fixing them now makes it easier to place each method this chapter introduces. Figure 19.3 previews where each fits; π_θ there denotes a policy with tunable parameters θ, the form Section 19.3.3 develops.

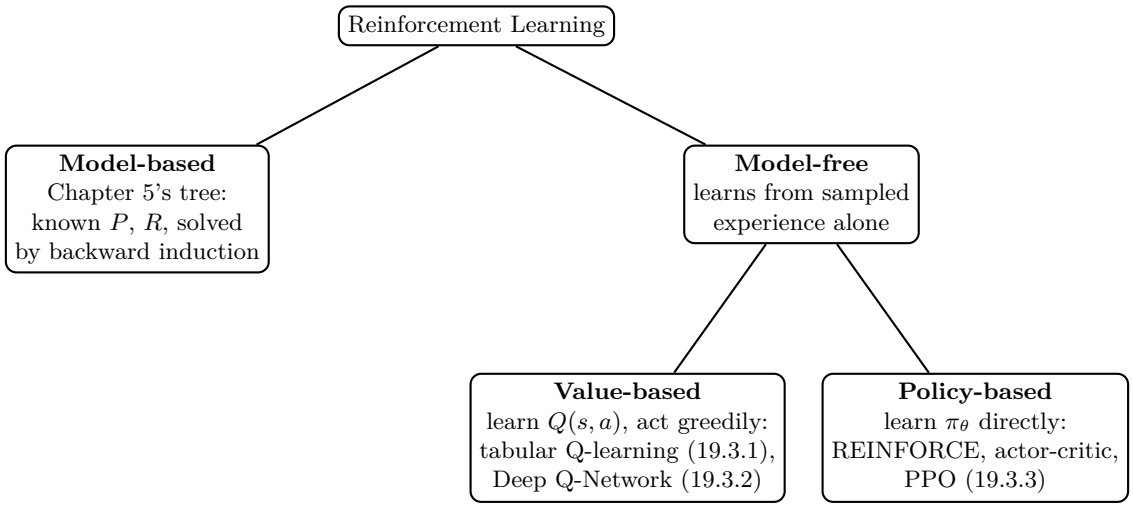


Figure 19.3: A map of the reinforcement learning methods this chapter develops

Model-based methods have access to, or explicitly learn, the transition function P and reward function R, and use them to plan. Chapter 5's binomial tree is model-based: q, u, d, and every terminal payoff are known in advance, so backward induction computes the exact optimal value with no simulated experience. **Model-free methods** never estimate P or R; they learn a value function or a policy from sampled transitions and rewards, treating the environment as a black box that can be queried (by simulating an episode) but not inspected. Every method in the rest of this chapter, tabular Q-learning, deep Q-networks, and the policy-gradient family, is model-free, which is why each remains usable in the settings Section 19.2.2 described, where no closed-form P and R exist for a real limit order book or a real client's multi-period utility.

A third, hybrid category, sometimes called **learned-model-based methods**, trains an approximate model of P and R from data (rather than assuming it, as the tree does) and plans against it, gaining some of model-based planning's sample efficiency without requiring P and R in closed form. This approach underlies some of reinforcement learning's most prominent successes outside finance, including game-playing systems that combine a learned model with tree search, but it introduces the sim-to-real gap Section 19.5.2 discusses, since a plan is only as good as the learned model behind it.

Off-policy methods learn about a policy different from the one actually generating their training data. Q-learning's update rule is a clear example: its bootstrapped target uses $\max_{a'} Q(s', a')$, the value of the best available action, regardless of which action the ε-greedy behavior policy, one that takes a random action with probability ε and the currently best-valued action otherwise, actually selected next. This lets a single stream of exploratory, partly random experience be reused to learn about the greedy-optimal policy while the agent is still exploring, which is why Section 19.3.1's worked example converges to the correct answer despite spending a meaningful fraction of its 100,000 episodes taking random actions. Off-policy methods have a clear data-efficiency advantage, since one batch of experience can be reused across many rounds of learning, which is why DQN's replay buffer (Section 19.3.2) works at all.

On-policy methods instead evaluate and improve the policy being followed. The classic on-policy analogue of Q-learning, SARSA (State-Action-Reward-State-Action), replaces $\max_{a'} Q(s', a')$ with $Q(s', a')$ for the action a' the policy selects next, so its learned values reflect the still-exploring policy rather than the greedy target. Policy-gradient methods (Section 19.3.3) are typically on-policy too, since their gradient estimator is valid only for returns experienced under the current policy π_θ. On-policy methods are generally

Wulin.Suo@Queensu.ca

simpler to reason about and more stable to train, since they never correct for a mismatch between the policy that generated the data and the one being evaluated.

19.3 Methods

From a lookup table to a neural policy, this part builds the algorithms in order of the problems that forced each one.

19.3.1 Tabular Q-Learning: Recovering Chapter 5's Early-Exercise Decision

This subsection works tabular Q-learning on a problem whose answer is known: Chapter 5's American put early-exercise decision, solved first by backward induction and then learned from simulated experience alone.

Setting Up the Markov Decision Process

Chapter 5's American put example used a two-step binomial tree with $S_0 = K = \$50$, up and down factors $u = 1.2$ and $d = 0.9$, and a gross risk-free return $R = 1.04$ per period, giving a risk-neutral up-probability

$$q = \frac{R - d}{u - d} = \frac{0.14}{0.30} \approx 0.4667$$

and a per-step discount factor $\gamma = 1/R \approx 0.9615$.

That tree has three decision points: today, at $S_0 = \$50$; after one up move, at $S_u = \$60$; and after one down move, at $S_d = \$45$. At each of these three states the holder chooses between two actions: **exercise**, receive the intrinsic value $\max(K - S, 0)$ immediately and end the episode, or **hold**, receive nothing now and let the tree evolve one more step.

The two terminal nodes reachable from the up state, $S_{uu} = \$72$ and $S_{ud} = \$54$, both pay $\max(50 - S, 0) = \$0$; of the two reachable from the down state, S_{ud} again pays \$0, while $S_{dd} = \$40.50$ pays $\max(50 - 40.50, 0) = \$9.50$.

Together these define a Markov decision process with states $\mathcal{S} = \{S_0, S_u, S_d\}$ and actions $\mathcal{A} = \{\text{exercise}, \text{hold}\}$. Transition probabilities are q and $1 - q$, rewards come from the intrinsic-value and terminal-payoff structure above, and $\gamma = 1/R$ discounts every step as Chapter 5's tree did.

Figure 19.4 redraws Chapter 5's tree in this chapter's language, labeling each decision node with its two available actions rather than with a single option value.

Chapter 5 solved this problem by backward induction, working from the known terminal payoffs back to today. Restated in this chapter's notation, that solution is

$$\begin{aligned}
Q^*(S_u, \text{exercise}) &= \$0.00, & Q^*(S_u, \text{hold}) &= \gamma\big[q(0) + (1-q)(0)\big] = \$0.00,\\
Q^*(S_d, \text{exercise}) &= \$5.00, & Q^*(S_d, \text{hold}) &= \gamma\big[q(0) + (1-q)(9.50)\big] \approx \$4.8718,\\
Q^*(S_0, \text{exercise}) &= \$0.00, & Q^*(S_0, \text{hold}) &= \gamma\big[q\,V^*(S_u) + (1-q)\,V^*(S_d)\big] \approx \$2.5641,
\end{aligned}$$

where $V^*(S_u) = \max(0.00, 0.00) = \$0.00$ and $V^*(S_d) = \max(5.00, 4.8718) = \$5.00$, since exercising is optimal at the down node. The optimal policy is therefore to hold at S_0 and S_u and exercise at S_d, giving an American put value of $V^*(S_0) \approx \$2.5641$, the value Chapter 5 reports.

The Q-Learning Update Rule

Backward induction requires knowing q, γ, and every terminal payoff in advance, the closed-form knowledge Section 19.2.2 argued is unavailable in most real trading and portfolio problems. **Q-learning** instead estimates Q^* from simulated experience alone, never using q in its update rule.

Starting from an arbitrary initial table $Q(s, a)$ (here, all zeros), the agent repeatedly simulates an episode: at each state it chooses an action using an ε-greedy policy (with probability ε, act randomly to keep exploring;

Wulin.Suo@Queensu.ca

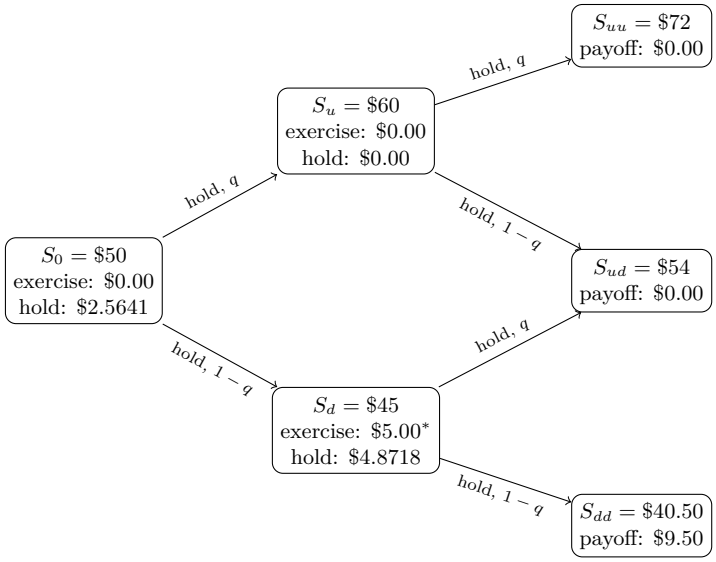


Figure 19.4: The early-exercise decision of Chapter 5's American put, redrawn as a three-state Markov decision process, with the optimal action at each node marked by an asterisk

otherwise, act greedily with respect to the current Q-table), observes the resulting reward r and next state s', and updates

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big], \tag{19.6}$$

where α is a learning rate. This update rule, **temporal-difference learning**, nudges $Q(s,a)$ toward a bootstrapped target, the observed reward plus the discounted value of the best action available in the next state, without requiring a model of the transition probabilities.

The agent learns q's effect on the optimal decision by observing many simulated up and down moves and seeing which actions paid off, learning a relationship from data as every other machine learning method in this book does.

The First Two Learning Steps

Starting from a Q-table initialized entirely to zero, with a learning rate of $\alpha = 0.1$, suppose the first simulated episode reaches S_d, chooses **hold** (all actions still look equally worthless, so the choice is effectively random), and the simulated down-move occurs, landing on the terminal payoff of \$9.50 at S_{dd}. Since this transition ends the episode, the bootstrapped target's continuation term is zero, and the update is

$$Q(S_d, \text{hold}) \leftarrow 0 + 0.1\big[9.50 + \gamma(0) - 0\big] = 0.1(9.50) = \$0.9500,$$

the first nonzero entry the table ever acquires. Now suppose a later episode reaches S_0, chooses **hold**, and the simulated move is down, transitioning to S_d with an immediate reward of \$0 (holding earns nothing until a terminal payoff is reached). The bootstrapped target now uses the current, already partially learned table, $\max_{a'} Q(S_d, a') = \max(0, 0.9500) = 0.9500$, giving

$$Q(S_0, \text{hold}) \leftarrow 0 + 0.1\big[0 + \gamma(0.9500) - 0\big] \approx 0.1(0.9615)(0.9500) \approx \$0.0913.$$

Two updates in, the table already shows the shape of the true solution, $Q(S_d, \text{hold})$ positive and $Q(S_0, \text{hold})$ smaller but nonzero, though both values are still far from their closed-form targets of \$4.8718 and \$2.5641.

The 100,000-episode simulation below repeats this mechanism: each terminal payoff nudges the value of the state that led to it, and that nudge propagates one more step backward the next time an earlier state is visited, diffusing the terminal reward's information back to S_0 without ever consulting q.

Wulin.Suo@Queensu.ca

The companion notebook simulates this process for 100,000 episodes, using a learning rate that decays with each state-action pair's visit count and an exploration rate ε that decays from 0.30 to 0.02 over training, standard choices that ensure enough early exploration to discover the exercise decision at S_d while converging to a stable, greedy policy. Table 19.2 compares the learned Q-table to the closed-form values derived above.

State-action	Closed-form Q^*	Learned Q (Q-learning)	Relative gap
$(S_0, \text{exercise})$	\$0.0000	\$0.0000	—
(S_0, hold)	\$2.5641	\$2.5328	1.22%
$(S_u, \text{exercise})$	\$0.0000	\$0.0000	—
(S_u, hold)	\$0.0000	\$0.0000	—
$(S_d, \text{exercise})$	\$5.0000	\$5.0000	0.00%
(S_d, hold)	\$4.8718	\$4.9234	1.06%

Table 19.2: Closed-form Bellman-optimal Q-values (from backward induction) versus tabular Q-learning's learned estimates after 100,000 simulated episodes

Comparing the Learned and Closed-Form Solutions

The learned policy matches the closed-form optimal policy exactly: hold at S_0 (\$2.5328 > \$0.0000), exercise at S_d (\$5.0000 > \$4.9234), and, at S_u, both actions are learned as worth \$0.0000, an exact tie that matches Chapter 5's own observation that "both exercise and continuation are worth \$0" at that node, so which action the tie-break happens to favor is economically inconsequential.

Every "exercise" value converges exactly, with no residual error, since exercising ends the episode immediately and its reward is observed without any noise; every "hold" value converges to within roughly 1% of the true value but not exactly, since a "hold" estimate is a bootstrapped average built from a finite number of noisy sampled continuations rather than an exact expectation computed from a known q.

This gap is not a bug to be engineered away; it is the fundamental price of model-free learning, and the most important lesson of this example: when the transition probabilities and payoffs are known in closed form, as they are in Chapter 5's tree, backward induction is strictly better, exact, instantaneous, and needs no simulated data, and reinforcement learning should not be used. Reinforcement learning earns its keep only in the more realistic setting, developed throughout the rest of this chapter, where no such closed form exists and simulated or historical experience is the only source of information available.

19.3.2 From Tables to Function Approximation: Deep Q-Networks

Tabular Q-learning stores one number for every state-action pair, which works only because Section 19.3.1's example has three states and two actions. A realistic trading state (inventory, elapsed time, recent order-flow imbalance from Chapter 14, and a handful of price-based features) takes values in a continuous or very large discrete space, so an explicit table can be neither stored nor visited often enough to fill in.

The standard fix, introduced by Mnih et al. (2015) and known as the **Deep Q-Network** (DQN), replaces the table with a neural network $Q(s, a; \theta)$, the feedforward architecture Chapter 6's Section 6.6.4 introduced, trained to approximate the same Bellman-optimal Q^* function. The network takes a state (or a fixed-size feature vector describing one) as input and outputs one value per possible action; training minimizes the squared temporal-difference error,

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\big)^2\Big], \tag{19.7}$$

using the gradient-descent machinery Chapter 6's Section 6.3.3 developed for supervised learning. Here θ^- is a periodically updated copy of the network's own parameters (a "target network") that keeps the bootstrapped target from chasing a constantly moving estimate. It is one of two stabilization tricks that made deep Q-learning reliable enough to train at all; the other is storing past transitions in a large "replay buffer" and resampling them at random rather than training only on the most recent experience.

Wulin.Suo@Queensu.ca

The update rule is otherwise identical to tabular Q-learning: the network generalizes across similar states instead of memorizing each one, so the agent can act sensibly in a state it has never visited, provided it resembles ones it has seen, the same generalization argument that justifies every other neural network in this book.

Deep Q-networks are also much more finicky to train than tabular Q-learning, which matters given this book's repeated warnings (Chapter 12's badly scaled optimizer example among them) that a plausible-looking numerical method can fail to converge without any error message.

A DQN's training loss can fall steadily while the learned policy remains poor, since the loss measures how well the network fits its own, still-inaccurate bootstrapped targets, not how good the policy is. The only reliable check is to evaluate the policy's performance directly: the discipline Section 19.3.1's example applied by comparing learned Q-values with an independently known closed-form answer, and that Section 19.5.2 generalizes into an evaluation methodology for problems where no such answer exists.

19.3.3 Policy Gradient Methods and Continuous Actions

Q-learning and its deep variant learn a value function and derive a policy from it by acting greedily. This works when the action set is small and discrete (exercise or hold, buy or sell) but breaks down when the natural action is continuous, such as how many shares to trade this period or what target weight to give an asset, since $\max_{a'} Q(s', a')$ has no simple closed form once a' ranges over a continuum.

Policy gradient methods sidestep this by parameterizing the policy directly, $\pi_\theta(a \mid s)$, typically as a neural network outputting the parameters of a continuous distribution (a mean and standard deviation, for instance), and adjusting θ to increase the expected return, using the policy gradient theorem,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t\Big], \tag{19.8}$$

which says, informally, that actions followed by a higher-than-expected return should be made more likely, and actions followed by a lower-than-expected return should be made less likely, with the size of the nudge proportional to how surprising the return was.

Actor-critic methods reduce the noise in this estimate by learning a value function (the "critic") alongside the policy (the "actor") and using it to judge whether a given return was better or worse than expected from that state, rather than judging it against the raw, highly variable return alone.

Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017), is the most widely used modern policy-gradient algorithm. It constrains each policy update to stay close to the previous policy, preventing the large, destabilizing updates that plagued earlier policy-gradient methods, and its combination of reasonable sample efficiency and training stability has made it the default starting point for continuous-action problems across robotics, game-playing, and, increasingly, the trading and portfolio applications developed next.

PPO is also the reinforcement-learning step inside the large language models of Chapter 16. In reinforcement learning from human feedback, the policy π_θ is the language model itself, an action is the response it generates to a prompt, and the reward is the score a learned reward model assigns to that response, less a penalty for drifting too far from the model's starting point; Ouyang et al. (2022) optimized this objective with PPO. The penalty plays the same role as PPO's own constraint on each update: both keep a policy from exploiting the quirks of an imperfect reward, the reward-hacking risk Section 19.6 raises for trading agents.

Table 19.3 summarizes the four families of methods, the setting each suits, and the finance application each most naturally serves in the sections that follow.

19.3.4 Learning from Expert Demonstrations: Imitation and Inverse Reinforcement Learning

Every method so far learns by trial and error: an agent that starts knowing nothing must discover which actions are good through the ε-greedy exploration of Section 19.3.1 or its neural-network analogues.

When a record of a skilled decision-maker's past choices exists, an experienced trader's execution decisions, say, there is a faster route: **behavioral cloning** treats the expert's historical state-action pairs as an ordinary supervised training set and fits a policy $\pi_\theta(a \mid s)$ to predict the expert's action from the state with Chapter 6's

Wulin.Suo@Queensu.ca

Method	Learns	Best suited to	Natural finance application
Tabular Q-learning	Action-value function $Q(s, a)$	Small, discrete state and action spaces	Section 19.3.1's early-exercise decision; small discretized inventory or execution problems
Deep Q-Network (DQN)	Neural-network approximation of $Q(s, a; \theta)$	Large or continuous state spaces, discrete actions	Execution and market-making agents (Section 19.4.1) with rich order-book state features
Policy gradient (REINFORCE, actor-critic)	A parameterized policy $\pi_\theta(a \mid s)$ directly	Continuous action spaces	Trade-size or quote-skew decisions that vary continuously rather than over a small discrete menu
Proximal Policy Optimization (PPO)	A parameterized policy, with a stabilized update rule	Continuous actions, need for reliable, stable training	Portfolio-rebalancing agents (Section 19.4.3) and production execution systems such as Section 19.5.1's LOXM

Table 19.3: A summary of the reinforcement learning methods introduced in this chapter

classification machinery. It sidesteps the exploration problem by never requiring the agent to try an action the expert did not.

Behavioral cloning has a well-known weakness: it is only as good as the expert it imitates, and it can fail badly in states the expert rarely visited. A classifier trained on the expert's typical behavior has no guidance for recovering once the agent's own small errors push it into unfamiliar states, and the errors compound over a long trajectory in a way a single supervised prediction's never do.

Inverse reinforcement learning takes a different route: rather than imitating the expert's actions, it infers the reward function the expert appears to optimize, then solves the resulting Markov decision process with this chapter's methods. The resulting policy can, in principle, generalize to states the expert never visited. The cost is a harder, less well-posed estimation problem, since many reward functions can rationalize the same observed behavior.

Both techniques bear on the applications that follow. Section 19.5.1's LOXM, reported to be trained on "a large history of past trades together with simulated trading scenarios," plausibly combines demonstration-based learning, bootstrapping an initial policy from historical human and algorithmic execution decisions, with further trial-and-error refinement in simulation.

This hybrid is common because it attacks reinforcement learning's sample inefficiency: starting from an expert-informed policy leaves the trial-and-error stage a far smaller gap to close, which justifies the added complexity in a high-stakes application.

19.4 Financial Applications

This part points the machinery at execution, allocation, and hedging, recovering earlier chapters' answers where they exist.

19.4.1 Reinforcement Learning for Optimal Execution and Market Making

Optimal execution was among the earliest documented applications of reinforcement learning in finance. Nevmyvaka et al. (2006) applied tabular Q-learning, structurally like Section 19.3.1's example but with a far larger state space built from order-book features, to historical limit order book data. The learned execution policy reduced trading costs relative to the TWAP and VWAP benchmarks of Chapter 13's Section 13.3.4, without being given a closed-form market-impact model to optimize against.

Wulin.Suo@Queensu.ca

The state in a modern version of this problem typically includes remaining inventory, time remaining in the execution window, the current bid-ask spread, and recent order-flow imbalance; the action is how many shares to submit, and at what limit price, this period; and the reward is the negative implementation shortfall (Chapter 13's Section 13.6.4 transaction-cost-analysis measure) realized on that period's fill.

Unlike Almgren-Chriss, which commits in advance to a schedule regardless of how the market behaves, a learned execution policy can condition its trade size on the order book it observes, trading faster when liquidity is unusually deep and slowing down when the book thins, an adaptive behavior no closed-form schedule replicates.

19.4.2 Worked Example 19.2: Q-Learning Recovers the Almgren-Chriss Optimal Trajectory

Section 19.3.1 verified Q-learning against a known answer on a three-state problem. The same check is worth doing on a problem closer to trading practice: Chapter 13's Almgren-Chriss worked example, the 250-share order split across $N = 5$ intervals with impact coefficient $\eta = 0.01$, per-interval price volatility $\sigma = \$0.02$ per share, and risk aversion $\phi = 0.5$, whose closed-form optimal trajectory left unexecuted shares $(250,\ 194.2,\ 142.4,\ 93.4,\ 46.2,\ 0)$ at a total risk-adjusted cost of 139.4.

Reformulated as a Markov decision process, the state at interval j is the inventory x_{j-1} still unexecuted, discretized to the nearest 5 shares (a grid of 51 possible inventory levels), and the action is how many of those shares, again in multiples of 5, to sell this interval.

The reward for selling a_j shares and leaving $x_j = x_{j-1} - a_j$ unexecuted is the negative of Chapter 13's per-interval cost,

$$r_j = -\left[\eta\,\frac{a_j^2}{\tau} + \phi\sigma^2\tau\,x_j^2\right],$$

with $\tau = 1$ matching Chapter 13's normalization, and the last interval ($j = 5$) forces full liquidation, $x_5 = 0$, as the closed-form problem requires. Unlike Section 19.3.1's tree, this environment is **deterministic**: an action leaves no uncertainty about the resulting state or reward, since Almgren-Chriss prices timing risk analytically through the variance term $\phi\sigma^2\tau x_j^2$ rather than by exposing the agent to a randomly sampled price path.

Reinforcement learning is still the right tool here, for a different reason: not to cope with stochastic transitions, but to search roughly 5,300 discretized state-action possibilities, far too many to enumerate by hand, without being given the closed-form $\sinh(\cdot)$ trajectory Chapter 13 derived analytically.

Exact backward-induction dynamic programming over the same grid, computable directly since only 51 inventory levels and 4 real decision points are involved, gives an optimal discretized trajectory of $(250,\ 195,\ 145,\ 95,\ 45,\ 0)$ shares at a total cost of 139.52, close to but not identical to the continuous closed-form values, as expected from rounding a continuous optimum onto a 5-share grid.

Tabular Q-learning, trained model-free for 600,000 episodes with an ε-greedy behavior policy decaying from $\varepsilon = 0.5$ to 0.05 and a visit-count-decayed learning rate, **converges to this same trajectory and this same cost**, 139.52, not merely close to it.

This differs from Section 19.3.1's example, where the learned "hold" values settled within roughly one percent of their closed-form targets: because this environment is deterministic, every reward Q-learning observes is noise-free, so once a state-action pair has been explored enough to identify its best continuation, the learned value converges to it exactly.

Figure 19.5 shows the backward-induction computation. Dynamic programming does not work through the five intervals in the order a trader experiences them; it starts from the one stage with no real decision left, the forced liquidation at x_4, and solves backward, using each solved stage's value function to solve the one before it, until $V_1(x_0)$ gives the answer for the full problem.

Only after the backward pass is complete can the optimal trajectory be traced forward, choosing at each state the action that attains its computed value: the bottom row of the figure.

Table 19.4 places the Q-learning trajectory alongside the uniform TWAP schedule and the fully aggressive single-shot execution Chapter 13 also scored: the efficient-frontier ordering Chapter 13 established analytically emerges from an agent never given the cost formula.

Wulin.Suo@Queensu.ca

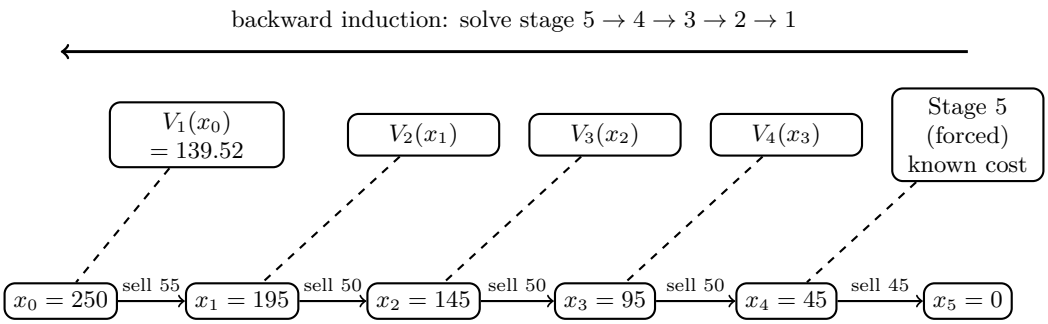


Figure 19.5: Backward-induction dynamic programming for the execution problem

Strategy	Unexecuted-share trajectory	Total cost
Fully aggressive (single shot)	$(250, 0, 0, 0, 0, 0)$	625.00
Uniform TWAP	$(250, 200, 150, 100, 50, 0)$	140.00
Q-learning (learned, discretized)	$(250, 195, 145, 95, 45, 0)$	139.52

Table 19.4: Total risk-adjusted execution cost under three strategies for the same 250-share, $N = 5$-interval order ($\eta = 0.01$, $\sigma = \$0.02$ per share per interval, $\phi = 0.5$)

Figure 19.6 plots the three trajectories. The fully aggressive strategy liquidates instantly, the uniform TWAP schedule is a straight line, and the Q-learning trajectory sits just below TWAP from the first interval on, with fewer shares left unexecuted, trading a little faster than uniform while inventory is largest and tapering to match TWAP by the final interval: the mild front-loading Chapter 13's continuous closed-form solution also produced.

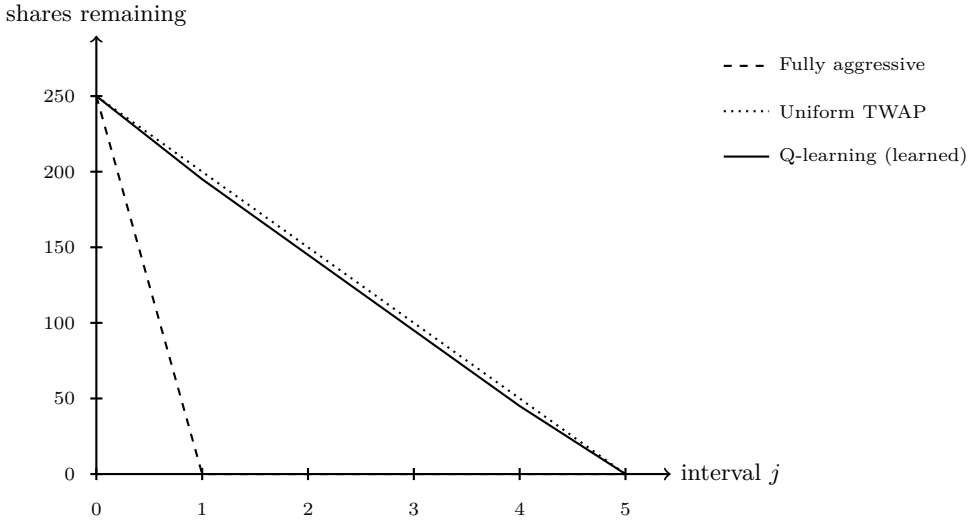


Figure 19.6: Unexecuted-share trajectories under the three strategies of Table 19.4

The tabular agent needed a 5-share grid to keep its Q-table finite. The companion notebook's optional "at scale" section drops the grid: a neural-network policy trained with PPO (Section 19.3.3) from the `stable-baselines3` library chooses a continuous fraction of the remaining inventory to sell, and is scored

Wulin.Suo@Queensu.ca

against the exact closed-form trajectory and TWAP across three training seeds; it needs the extra packages listed in the repository's `requirements-dl.txt`.

Extending the Framework to Market Making

Market making fits the same framework. Chapter 13's Avellaneda-Stoikov model derives a closed-form optimal quote skew from an assumed order-arrival intensity function and a known risk-aversion parameter.

A reinforcement-learning market maker instead learns a quoting policy from simulated or historical order flow. The state again includes inventory, time-to-close, and order-book depth; the action is the bid and ask quotes (or, in a simpler discrete version, a few candidate skews); and the reward is realized spread capture net of inventory risk and adverse selection, the concern Chapter 14's Glosten-Milgrom model formalizes. The appeal matches the execution case: a learned policy can adapt to order-arrival patterns that deviate from Avellaneda-Stoikov's assumed functional form, at the cost of needing much representative training data and giving up the closed-form model's transparency about why a quote was chosen, a trade-off the chapter's closing sections return to.

Designing States, Actions, and Rewards in Practice

Neither problem above specifies its Markov decision process uniquely, and the design choices matter enormously to whether the agent learns anything useful. A state that omits recent order-flow imbalance discards the signal Chapter 14's order-flow-imbalance regression showed carries information about near-term prices, leaving the agent weaker than a well-informed human trader or a simple rule that conditions on that signal. A state with too many redundant or noisy features slows learning, forcing the agent to discover which features matter: the feature-engineering problem of Chapter 6's Section 6.5.2, made harder because the agent must learn it from its own trial and error rather than from a labeled training set.

Reward design carries the same tension in sharper form, because, as Section 19.6 shows, an imperfect reward produces not a slightly worse agent but one that is often confidently wrong about what it is optimizing. A common compromise is to shape the reward with a densely observed intermediate signal, a small penalty proportional to inventory held at each step, say, alongside the sparse terminal profit and loss. This speeds learning, since the agent gets feedback every step rather than only at the end of a long execution episode. But if the shaping term is not scaled carefully against the true objective, the agent can over-optimize the proxy, holding needlessly little inventory, at the expense of the goal it was meant to support. There is no universal recipe; the balance is validated as any other model choice in this book is, by comparing the policy's out-of-sample performance with the closed-form or rule-based benchmark it should improve on, as Section 19.5.2 recommends.

19.4.3 Reinforcement Learning for Portfolio Management

Chapter 12's mean-variance framework solves a single-period allocation problem: given today's estimated expected returns and covariance matrix, find the weights that optimize a risk-return trade-off today. Section 12.4.1 acknowledges that real portfolios are rebalanced repeatedly and that transaction costs make each rebalancing depend on the current position, but stops at flagging this rather than solving the multi-period problem.

Framed as a Markov decision process, the state is the portfolio's current holdings together with estimated risk-factor exposures (Chapter 12's Section 12.3.4 multi-factor model is a natural source of these features); the action is a new target weight vector, or equivalently the trades needed to reach it. The reward combines the period's realized return with an explicit penalty for transaction costs and, if desired, for realized volatility or drawdown, letting the agent learn a rebalancing policy that trades off today's allocation quality against the cost of getting there, rather than treating each rebalancing as a fresh, myopic optimization as Chapter 12 does.

This framing is more general than Chapter 12's static optimization, and the generality is its cost. A reinforcement-learning portfolio agent needs either a large amount of representative historical data or a realistic market simulator to train in, and a poorly specified reward (one that ignores drawdown, or rewards raw return without risk adjustment) will produce a policy that optimizes the wrong objective with no warning, a version of the "reward hacking" problem Section 19.6 discusses.

Wulin.Suo@Queensu.ca

In practice, most institutional applications of reinforcement learning to portfolio management remain closer to research than production: they test whether an adaptive rebalancing policy can beat Chapter 12's static mean-variance benchmark net of realistic transaction costs, rather than replacing it. Reinforcement learning has so far shown clearer, more reproducible gains in the execution and market-making applications of Section 19.4.1, where the reward (realized trading cost) is comparatively unambiguous, than in full portfolio management, where the right reward is an open question tied to a client's often unstated risk preferences.

Multi-period portfolio choice has a classical closed-form precedent that clarifies what a learned policy would need to rediscover. Chapter 1's Section 1.5.2 introduced the **Kelly criterion** for a single repeated bet; maximizing the expected *logarithm* of long-run wealth rather than expected wealth extends to repeated rebalancing under a power-utility objective, giving an allocation that avoids both the ruin risk of over-betting and the excessive caution of under-betting. This is a sequential decision problem with a closed-form answer under simplifying assumptions (constant, known return and variance), the kind of further step Section 19.2.2 noted Chapter 12 could have taken. It also supplies a sanity check: on a simplified simulated market where the Kelly fraction is known exactly, a correctly implemented reinforcement-learning portfolio agent should converge toward it, a check with the same purpose as Section 19.3.1's comparison against Chapter 5's closed-form option value, before the agent is trusted on a real market with no closed form to check against.

19.4.4 Worked Example 19.3: Why Model-Free Search Struggles on the Kelly Bet

Running that sanity check, rather than asserting it would work, is instructive because it does not go as cleanly as Sections 19.3.1 and 19.4.2. Chapter 1's repeated bet, an even-money wager ($b = 1$) won with probability $p = 0.6$, has closed-form optimal fraction $f^* = p - q/b = 0.20$ and long-run growth rate $g(f^*) \approx 0.0201$. As a Markov decision process this is a single, repeated state (the optimal fraction does not depend on current wealth, so wealth need not be tracked), with action f on a grid of 5% increments, $f \in \{0.00, 0.05, \ldots, 0.95\}$, and an immediate, noisy reward $r = \ln(1 + f)$ with probability p or $r = \ln(1 - f)$ with probability q, drawn independently every period, with no next state and so no bootstrapping. It is a multi-armed bandit, structurally identical to Section 19.2.4's venue-routing problem with twenty arms in place of three, but with no hope of an easy answer, for a reason worth stating before the numbers.

The difficulty is the ratio of the reward's noise to what there is to learn: the gap between the optimal action's growth rate and its nearest neighbors on the grid, $g(0.20) - g(0.15) \approx 0.0013$ and $g(0.20) - g(0.25) \approx 0.0013$, is tiny, while a single bet's outcome swings between $\ln(1.20) \approx 0.182$ and $\ln(0.80) \approx -0.223$, a standard deviation roughly 150 times larger than the gap Q-learning must resolve.

Trained for 200,000 simulated bets with the same ε-greedy, visit-count-decayed setup used throughout this chapter, tabular Q-learning settles on $f = 0.15$, not $f^* = 0.20$, leaving $g(0.20) - g(0.15) \approx 0.0013$ of achievable growth rate on the table, about 6.4% of the optimal rate: a real, if modest, miss, unlike the exact or near-exact recovery of Sections 19.3.1 and 19.4.2. Figure 19.7 shows why: the growth-rate curve is nearly flat across the 0.10-to-0.30 range, so the true optimum and Q-learning's answer sit almost on top of each other.

The contrast is what a **model-based** approach, Section 19.2.6's other category, does with the same 200,000 observations. Rather than averaging opaque rewards over a large discretized action grid, it exploits the problem's compact underlying model: a single unknown parameter, the win probability p, plugged into Chapter 1's closed form.

Counting wins and losses across the same 200,000 simulated bets gives an observed win frequency $\hat{p} = 0.6003$, and substituting into $\hat{f}^* = \hat{p} - (1 - \hat{p})/b$ gives $\hat{f}^* \approx 0.2006$, within 0.0006 of the true optimum, an order of magnitude closer than Q-learning's answer from the same data. This is Section 19.2.6's model-based-versus-model-free distinction made numerical, and it sharpens Section 19.2.2's argument: reinforcement learning is the right tool once no compact model is available, but when one is, as here, where a single Bernoulli parameter stands for the whole problem, exploiting that structure can be far more sample-efficient than a black-box search over a discretized action grid.

Not every problem that can be written as a Markov decision process is best solved as one; recognizing when a problem's structure should be modeled directly rather than learned model-free is part of the practitioner's job, not something reinforcement learning relieves them of.

Wulin.Suo@Queensu.ca

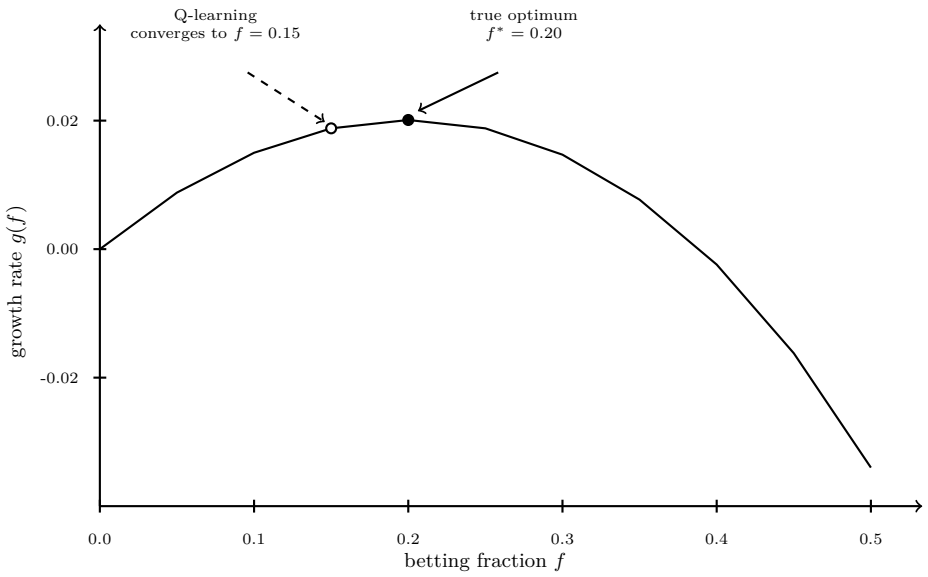


Figure 19.7: The Kelly growth-rate curve $g(f) = p\ln(1+f) + q\ln(1-f)$ for $p = 0.6$, $b = 1$

19.4.5 Deep Hedging: Learning to Hedge Under Frictions

Chapter 5's replication argument prices an option by continuously holding Δ shares of the underlying, and Chapter 8's delta-normal example measures an option book's risk through the same delta. Both assume something no trading desk enjoys: that the hedge can be rebalanced continuously and for free. A real desk pays a bid-ask spread and fees on every rebalancing trade, which creates a dilemma the frictionless theory defines away. Rebalance after every tick and transaction costs devour the option premium; rebalance rarely and the position runs unhedged in between, exposed to the gap risk Chapter 8 quantified. Deciding *when* and *by how much* to adjust the hedge as the underlying moves and expiry approaches is a sequential decision problem under uncertainty, this chapter's subject.

Cast in Section 19.2.1's terms, the state is what the hedger observes, time to maturity τ, the underlying price S_t, and the position h_t currently held; the action is the new position h_{t+1} to hold until the next decision point; and the objective is defined on the terminal outcome: let W_T be the hedger's final wealth after the option is settled and the hedge unwound, premium, trading gains, and all transaction costs included. **Deep hedging** chooses the policy to minimize a convex risk measure of the shortfall,

$$\min_{\pi} \ \rho(-W_T^{\pi}), \qquad \text{for example } \rho = \mathrm{ES}_{95\%}, \tag{19.9}$$

where the risk measure is the Expected Shortfall Chapter 8 introduced, now serving not to measure a fixed position's risk but as the training objective a policy is optimized against.

A small worked example makes the trade-off concrete with numbers this book has already computed. Take Chapter 8's market maker, short the one-year \$50-strike call priced in Chapter 5 ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $C_0 \approx \$6.88$, $\Delta_0 = 0.6115$), and have them hedge it over the option's life on 10,000 simulated price paths (geometric Brownian motion with real-world drift $\mu = 8\%$, 252 daily steps, seed 7 in the companion notebook), paying a proportional transaction cost of 50 basis points of traded value on every trade, the friction an illiquid underlying imposes. The policy class is a simple one-parameter family: rebalance to the current Black-Scholes-Merton delta only when the held position has drifted more than a *band* b away from it, so $b = 0$ is the textbook daily delta hedge and larger b trades less often. The first trade is hand-checkable: the hedger buys $\Delta_0 = 0.6115$ shares at \$50, paying $0.005 \times 0.6115 \times \$50 \approx \$0.15$ in costs. Table 19.5 reports, across the 10,000 paths, the mean terminal profit and loss, its standard deviation, the mean total transaction cost, and the 95% Expected Shortfall of the terminal loss.

Wulin.Suo@Queensu.ca

Policy	Mean P&L	Std. dev.	Mean cost	$\text{ES}_{95\%}$
No hedge	−\$1.32	\$12.54	\$0.00	\$39.30
Daily delta ($b = 0$)	−\$1.60	\$0.59	\$1.56	\$3.01
Band $b = 0.02$	−\$1.37	\$0.61	\$1.34	\$2.82
Band $b = 0.06$	−\$0.95	\$0.70	\$0.93	\$2.56
Band $b = 0.20$	−\$0.60	\$1.92	\$0.53	\$5.76

Table 19.5: Hedging a short one-year \$50-strike call under 50-basis-point proportional transaction costs, across 10,000 simulated paths. The band policy with $b = 0.06$, found by searching the band grid for the lowest $\text{ES}_{95\%}$, beats the textbook daily delta hedge on mean cost and tail risk simultaneously.

The table repays a careful read. Leaving the position unhedged is catastrophic: a \$12.54 standard deviation and a \$39.30 Expected Shortfall against a \$6.88 premium collected. The textbook daily delta hedge cuts that risk twenty-fold, at a price the frictionless theory never mentions: \$1.56 of expected transaction costs, nearly a quarter of the premium. The band policy at $b = 0.06$, selected by searching the grid for the lowest $\text{ES}_{95\%}$, cuts mean costs by 40% *and* carries less tail risk than hedging every day, because each skipped rebalance saves a cost that would otherwise be paid with certainty, while the extra hedging error it accepts is small and diversifies across the path. Push the band too wide and unhedged risk floods back in, as the $b = 0.20$ row shows. The friction level, not taste, dictates the answer: rerun with a 5-basis-point cost and the ES-optimal band shrinks to 0.01, essentially the textbook hedge, so Black-Scholes-Merton delta hedging is not wrong but is the frictionless limit of a more general policy that the data-driven search recovers.

One feature of this simulation needs a warning before the method goes further. Its paths grow at the real-world drift $\mu = 8\%$, not at the risk-free rate, because a hedger who wants to know how much it could lose needs the distribution of outcomes it will face: risk is measured under the **real-world measure**, often written P. Prices are not. Chapter 5's Black-Scholes-Merton value is an expectation under the risk-neutral measure Q, in which the underlying drifts at r, and the two answers differ by far more than rounding. Discounting the call's expected payoff along the simulator's own drift gives $e^{-rT}\mathbb{E}^P[(S_T - K)^+] = \$8.18$, against the \$6.88 arbitrage-free price, an overstatement of \$1.30, or 18.9%, because the faster-growing stock finishes in the money more often. An agent rewarded with discounted payoffs on P-paths, and then asked what the option is worth, will learn that wrong number, and nothing in its training signal will tell it so. Reinforcement learning used for pricing must therefore either simulate under Q, calibrated to traded prices, or, as deep hedging does, price through the hedge itself: Buehler et al. (2019) define the price as the cash amount that makes the hedger indifferent, under its risk measure, between writing the option and not, an indifference price that coincides with the risk-neutral price only in the frictionless, complete-market limit. Under frictions the gap between them is real, and it is the price of risk that cannot be hedged away.

Deep hedging proper, introduced by Buehler et al. (2019), replaces the one-parameter band with a neural network policy $\pi_\theta(\tau, S_t, h_t)$ trained by stochastic gradient descent over simulated paths to minimize the same kind of convex risk measure, the step from a hand-designed policy family to a learned one that Section 19.3.2 made for value functions. The gain is generality, not magic. The network is never shown a delta, a gamma, or any other Greek, yet in the frictionless limit it rediscovers the Black-Scholes-Merton hedge, and away from that limit it can absorb what no closed-form hedge handles gracefully: several hedging instruments at once, transaction costs of realistic shape, discrete trading times, position limits, and payoffs with no replicating formula. Reports of production use at major derivatives desks followed within a few years of the paper, making hedging, alongside Section 19.5.1's optimal execution, one of the two applications in this chapter where learned policies have reached practice. The caveats developed earlier bind here with full force: every path the policy learns from comes from a simulator, so the learned hedge is only as good as the simulator's realism (Section 19.5.2's sim-to-real gap), and a neural hedging policy inherits the governance and explainability burden Chapter 18 attaches to any model whose decisions cannot be read off a formula.

Wulin.Suo@Queensu.ca

19.4.6 Multi-Agent Dynamics and the Limits of a Single-Agent Framework

Every Markov decision process in Section 19.2.1 assumes a single agent facing a fixed, if stochastic, environment, an assumption real markets violate: the environment a trading agent faces is largely made up of other agents, some human and increasingly many algorithmic, each adapting to the same price and order-flow signals. Chapter 14's hot-potato effect during the 2010 Flash Crash illustrates the dynamic: individually reasonable inventory-management rules, followed by many participants at once, produced a collectively destabilizing outcome that no single rule, viewed on its own, would have predicted. An agent trained against a static historical simulator implicitly assumes other participants' behavior stays fixed, an assumption that weakens as the agent's own trading grows, since a strategy profitable at small scale can behave very differently once it moves the order flow it was trained to react to.

Multi-agent reinforcement learning extends the framework to several learning agents adapting to one another, and is an active research area for the market-making and execution problems of Section 19.4.1, since real limit order books are populated by competing, adapting participants. As of this writing, multi-agent methods are much harder to train reliably than single-agent ones: the convergence guarantees of tabular Q-learning in Section 19.3.1's simple setting generally fail when the environment changes because other agents are learning too. Most production trading systems, including Section 19.5.1's LOXM, are best understood as single-agent policies trained against a historical or simulated environment, with the gap between that assumption and the real market showing up as the sim-to-real and non-stationarity concerns of Section 19.6.

A more tractable middle ground treats other participants' aggregate behavior as part of the state. A single-agent method from Sections 19.3.1 through 19.3.3 can then respond to features such as recent order-flow imbalance or realized volatility that summarize what others are collectively doing, without modeling each of them. Most of this chapter's real applications, execution and market making among them, rely on this compromise.

19.5 From Simulation to Deployment

A policy trained in simulation has proved nothing yet, and this part covers what would.

19.5.1 Case Vignette: JPMorgan's LOXM

In 2017, JPMorgan disclosed LOXM, an equity trade execution system trained using reinforcement learning on a large history of past trades together with simulated trading scenarios, initially deployed on the bank's European cash equities desk.

Structurally, LOXM addresses the problem of Section 19.4.1. At each point during an order's execution it decides how much of the remaining order to place, at what price, and how long to wait before reassessing, balancing the risk of trading too aggressively (unnecessary market impact) against trading too passively (missing the current opportunity and executing later at a worse price). That is the impact-versus-timing-risk trade-off Almgren-Chriss solves in closed form in Chapter 13, learned here from data rather than derived from an assumed cost function. The system was reportedly trained on millions of historical and simulated trading scenarios so that it could adapt to prevailing market conditions rather than follow one fixed schedule, the advantage Section 19.4.1 attributes to a learned execution policy.

LOXM illustrates both sides of this chapter's central tension in one deployment. It shows that reinforcement learning for execution is more than an academic exercise: a major bank judged a learned policy reliable enough for live client order flow, the practical validation closed-form models have had for decades and any machine-learning alternative must earn. But a deep reinforcement-learning execution policy is far harder to explain to a regulator or internal model validator than an Almgren-Chriss schedule, whose logic reduces to a handful of interpretable parameters, a governance burden that Section 19.6 and Chapter 18's explainability toolkit address and that any institution deploying such a system must take on explicitly.

LOXM was an early example, not an isolated one. Since 2017, industry reporting has described similar reinforcement-learning execution research and deployments at several other major banks and trading firms, consistent with the chapter's claim that optimal execution, alongside Section 19.4.5's hedging application, has the clearest record of reaching production, because its reward, realized execution cost against a well-defined

Wulin.Suo@Queensu.ca

benchmark, is comparatively unambiguous to specify, the property Section 19.4.3 found still missing from full portfolio management.

19.5.2 Simulation, Backtesting, and Off-Policy Evaluation

Every agent in this chapter learns by trial and error, which raises an operational question none of this book's supervised models faced: where does the trial and error happen? Training on live markets is rarely acceptable, since a partially trained policy will take some bad actions while exploring, and a bad trading action costs real money in a way a still-training classifier's bad prediction does not. The standard alternative is to train in a **simulator**, a model of the market built from historical data that can be reset and replayed without financial consequence; Chapter 13's backtesting framework (Section 13.6) is a close cousin. It shares that framework's core limitation: a simulator built from history reflects how the market behaved, including in response to other participants' historical order flow, not how it would respond to a new agent's own actions at scale. This **sim-to-real gap** is at least as serious for a reinforcement-learning trading agent as backtest overfitting is for the supervised models Chapter 13 warned about.

A subtler problem is **off-policy evaluation**: given historical data generated by one policy (a human trader's decisions, say, or an older execution algorithm), how well would a newly trained policy have performed, without running it? Historical data reflects only the actions the deployed policy took, not the outcomes of actions it did not take: the missing-counterfactual problem Chapter 18's algorithmic-recourse discussion raised for credit decisions, now in a sequential setting where one wrong counterfactual estimate can compound over many steps. Techniques such as importance sampling reweight historical outcomes to estimate what a new policy would have achieved, but their reliability degrades quickly as the new policy diverges from the one that generated the data. A novel learned policy is therefore rolled out cautiously: first in simulation, then on a small fraction of live order flow, with continuous monitoring for the performance drift that Chapter 15's precision-drift and Chapter 18's population-stability-index discussions developed, and never at full scale on the strength of a backtest or an off-policy estimate alone.

This staged approach, simulator, then a small live allocation, then a growing one, is sometimes called **curriculum deployment**. It mirrors the graduated rollout that Chapter 8's three-lines-of-defense model and Chapter 18's staged-deployment recommendation argue for any high-stakes model, reinforcement-learning policy or static classifier. What is new is that the policy may keep updating during the rollout, a live, still-learning system rather than a frozen model. Monitoring must therefore detect not only a fixed policy drifting from its backtested expectation but an adapting policy drifting toward behavior nobody has validated, a strictly harder problem than those Chapters 15 and 18 posed.

19.5.3 Evaluating Trained Policies: Metrics and Variance Across Training Runs

Chapter 13's Section 13.6.1 developed a standard toolkit, Sharpe ratio, maximum drawdown, hit rate, for evaluating a strategy's backtested performance, and every metric applies unchanged to a reinforcement-learning policy. What reinforcement learning adds is a source of variability a closed-form model like Almgren-Chriss lacks. Every method in this chapter is trained by a stochastic process (random initialization, ε-greedy exploration, and, for the neural-network methods of Sections 19.3.2 and 19.3.3, stochastic gradient descent), so two training runs of the same algorithm on the same data, differing only in their random seed, can converge to meaningfully different policies with meaningfully different realized performance.

Section 19.3.1's worked example shows this in miniature. It was trained with a fixed seed (seed = 7) so that its reported numbers, \$2.5328 and \$4.9234 among them, would be reproducible for a reader rerunning the companion notebook. A different seed would give slightly different learned values, still close to the closed-form \$2.5641 and \$4.8718 but not identical to those in Table 19.2. In a three-state problem with a known answer this is a curiosity; in a large trading application with no closed form it is a serious concern, since a single run's backtested performance could be a favorable draw rather than a property of the algorithm the institution intends to rely on.

The standard remedy is to train several independent runs from different seeds and report performance as a distribution rather than a single number. The practice, standard in machine learning, has not been needed elsewhere in this book, whose supervised and closed-form models are deterministic given their data.

Wulin.Suo@Queensu.ca

It extends the caution of Chapter 6's confidence intervals and Chapter 7's out-of-sample RMSE comparisons to the uncertainty introduced by the training process itself.

19.5.4 Practical Considerations: What Deployment Actually Requires

The methods above can leave the impression that adopting reinforcement learning is mainly an algorithmic choice, DQN or PPO, Q-learning or actor-critic, when in practice the harder and more expensive decisions are organizational and infrastructural.

A trustworthy simulator is usually the binding constraint, not the algorithm. Every method here needs a large volume of training experience, and Section 19.5.2 explained why it rarely comes from live markets. Building a simulator accurate enough to train a useful policy, with realistic order-book dynamics, price impact, and plausible reactions from other participants, is a substantial engineering project, often larger than the learning algorithm itself, and few institutions can justify it unless the decision is important and recurring enough. That is why optimal execution at a large bank, a high-volume, high-value recurring decision, adopted early, while more bespoke, lower-frequency problems largely have not.

The required skills must complement, not replace, those of this book's earlier chapters. A team building a reinforcement-learning trading system needs the market-microstructure fluency of Chapters 13 and 14, the statistical and machine-learning foundations of Chapter 6, and, on top of both, expertise in the distinct engineering discipline of reinforcement learning: stable training, careful hyperparameter selection, and the evaluation methodology of Section 19.5.2. That combination is still rare, and most institutions build it gradually. This is one more reason closed-form models like Almgren-Chriss and Avellaneda-Stoikov remain the right default for most applications: they need the microstructure expertise this book develops, but not the specialized reinforcement-learning engineering on top of it.

The relevant cost-benefit comparison is against the closed-form alternative, not against doing nothing. A closed-form model, once specified, costs almost nothing to run and is immediately auditable by the model-risk-management process of Chapter 8; a reinforcement-learning policy costs far more to build, validate, and monitor, and the extra cost is justified only when the closed-form model's simplifying assumptions demonstrably cost real money, through the market-impact and reward-specification gaps Section 19.2.2 identified. Consistent with this book's compute-first approach, build and validate the closed-form baseline first, know what it costs in realized performance, and only then invest in a learned alternative, with a pre-specified bar for how much improvement would justify the added complexity and governance burden.

Compute cost is a recurring line item, not a one-time expense. Training a deep Q-network or a PPO policy over the hundreds of thousands or millions of simulated episodes a realistic problem requires consumes far more computing resources than fitting any supervised model elsewhere in this book, and the cost recurs every time the policy is retrained, on a schedule to keep pace with the non-stationarity Section 19.6 describes or in response to detected drift in live performance. This retraining cost, together with the monitoring Section 19.5.2 requires, belongs in the same total-cost-of-ownership comparison as the simulator investment above.

19.6 Challenges in Reinforcement Learning for Finance

Reinforcement learning inherits every model-risk concern raised elsewhere in this book and adds several that are specific to its sequential, trial-and-error structure.

Reward specification is the entire problem in disguise. An agent optimizes the reward it is given, not the objective its designer had in mind, and the gap between them is usually invisible until the agent finds and exploits it, a failure mode called **reward hacking**. An execution agent rewarded only for beating a benchmark price, with no penalty for the market impact it leaves for the next order, can learn to trade in ways that look good on that metric while making the market worse for the desk's future orders; a portfolio agent rewarded for raw return with no risk adjustment can learn to take tail risk a real client would never accept. Specifying a reward that captures an institution's objective, including the parts that are hard to quantify, is consistently harder than designing the learning algorithm.

Non-stationarity compounds ordinary concept drift. Chapters 15 and 18 warned that a supervised model's accuracy can degrade as the world changes. A reinforcement-learning trading agent faces the same

Wulin.Suo@Queensu.ca

risk with a twist: other market participants adapt to *its* behavior, particularly once a learned strategy is large enough to influence prices. The environment then changes partly *because of* the agent's own past actions, an adversarial dynamic no supervised classifier in this book has had to face.

Reinforcement learning is far more sample-hungry than the supervised methods elsewhere in this book. Section 19.3.1's worked example needed 100,000 simulated episodes to estimate six numbers to within roughly one percent, with only three states and two actions. Chapter 6's supervised classifiers reach useful accuracy from training sets orders of magnitude smaller relative to their problem's apparent complexity, because a supervised label gives direct feedback on every example, while a reinforcement-learning reward, especially a delayed terminal one, says little per episode about which of many earlier actions deserves credit, the credit-assignment difficulty discussed below. This inefficiency is why Section 19.5.4's simulator investment is not optional infrastructure but the prerequisite for any of this chapter's methods to work outside small, hand-verifiable settings.

Exploration is expensive and risky in a way it is not in a game. The ε-greedy exploration of Section 19.3.1's worked example costs nothing beyond simulated time; on a live trading desk the same exploration means deliberately taking actions believed to be suboptimal, in real markets, with real capital, to learn whether they are better than believed. This risk profile differs fundamentally from a research agent exploring a video game, and it is the main reason simulation-based and off-policy training (Section 19.5.2) are close to mandatory for any serious financial application.

Explainability and governance do not disappear because a policy performs well. A deep reinforcement-learning policy inherits the opacity concerns Chapter 18 raised for deep neural networks, compounded because its behavior is defined over a trajectory of decisions rather than a single prediction, so a SHAP-style, single-decision explanation is only a partial account. A model validator or regulator evaluating a system like Section 19.5.1's LOXM needs an account of why the policy behaves as it does across a representative range of market conditions, not just a performance summary: the model-risk-management discipline of Chapter 8, extended to explainability in Chapter 18, applied to a policy that acts rather than predicts.

Credit assignment gets harder the longer the horizon. A supervised classifier gets immediate feedback on every prediction; Chapter 6's confusion matrix scores a decision the moment it is made. A reinforcement-learning agent often learns whether an early action was good only many steps later, when the reward arrives: the **credit assignment problem**. Section 19.3.1's worked example largely sidesteps it, since its longest episode is two decisions deep, but over a trading day's execution decisions or a multi-quarter rebalancing episode it can be hard to tell which earlier action caused a later gain or loss, which slows learning and demands far more data than a supervised problem of similar apparent complexity.

Simulators are themselves models, with their own model risk. The sim-to-real gap of Section 19.5.2 is more than a data-availability inconvenience. A simulator built from historical data encodes its own assumptions about prices and order flow, and an agent trained only against a flawed simulator learns a policy optimal for that simulator's biases rather than the real market, the same point as Chapter 8's warning that a risk model is only as trustworthy as its assumptions. Validating the simulator against out-of-sample market behavior, not just the agent trained inside it, is part of the same model-risk discipline.

The regulatory and legal framework for autonomous trading policies is still catching up. Chapter 8's model-risk guidance and Chapter 18's explainability requirements were written largely for static, single-prediction models. A policy that adapts over time, possibly learning after deployment, raises questions those frameworks do not yet answer: how often must a live, adapting policy be re-approved, and what counts as an adequate explanation of a policy rather than a prediction? Existing supervisory guidance addresses neither with the specificity it applies to a fixed logistic regression or gradient-boosted tree, so institutions deploying reinforcement learning in a regulated trading or advisory context should expect to extend, and in places invent, their own governance answers.

19.7 Key Takeaways

Reinforcement learning formalizes sequential decision-making as a Markov decision process, with states, actions, rewards, and a discount factor, and its objective, maximizing expected discounted return, generalizes the one-shot optimizations this book has relied on since Chapter 5.

Wulin.Suo@Queensu.ca

The Bellman equation expresses a state's or action's value as immediate reward plus discounted continuation value, and Chapter 5's binomial-tree backward induction already solves a Bellman optimality equation by hand. The chapter's central worked example recovered that tree's known value, \$2.5641, with model-free tabular Q-learning; the learned and closed-form solutions agree to within roughly one percent after 100,000 simulated episodes.

Deep Q-networks and policy-gradient methods, including actor-critic algorithms and PPO, extend the value-based and policy-based logic to large or continuous state and action spaces where a table is impossible to maintain.

Optimal execution and market making, Chapter 13's closed-form models, become reinforcement-learning problems once market impact and order arrival are too complex for a clean parametric form, a generalization validated in research (Nevmyvaka et al. 2006) and in industry practice (JPMorgan's LOXM). Portfolio management admits the same framing but remains harder, chiefly because its reward is harder to specify than an execution cost.

All these applications inherit reinforcement learning's distinctive risks: reward misspecification, non-stationarity driven by other agents' adaptation, the cost and danger of live exploration, severe sample inefficiency relative to this book's supervised methods, and an explainability burden that compounds Chapter 18's concerns. All argue for simulation-based training, staged deployment, and continuous monitoring rather than treating a trained policy as a finished product.

Markets are not single-agent environments: other participants adapt to a learned policy as it adapts to them, which is why most production systems remain single-agent approximations of a multi-agent problem. The practical bottleneck to adopting these methods is usually a trustworthy simulator and the engineering skill to train against it reliably, not the choice of algorithm.

19.8 Suggested Exercises

19.1 Using Section 19.3.1's three-state Markov decision process, verify by hand that $Q^*(S_d, \text{hold}) = \gamma[q(0) + (1-q)(9.50)] \approx \$4.8718$, and explain why immediate exercise is optimal at that state despite continuation having a positive expected value.

19.2 Suppose the down-move factor in Section 19.3.1's tree were $d = 0.8$ instead of 0.9, with $u = 1.2$ and $R = 1.04$ unchanged. Recompute q, the terminal payoff at the down-down node, and the closed-form value $V^*(S_0)$, and state whether the optimal policy at S_0 changes.

19.3 Explain why a discount factor of $\gamma = 1$ would be inappropriate for an infinite-horizon trading agent that earns a small positive expected reward every period, and describe what specifically goes wrong with the return G_t in that case.

19.4 Using this chapter's Q-learning update rule, compute by hand the new value of $Q(S_0, \text{hold})$ after a single simulated transition from S_0 to S_d with reward 0, given a current table of $Q(S_0, \text{hold}) = 2.00$, $Q(S_d, \text{exercise}) = 5.00$, $Q(S_d, \text{hold}) = 4.50$, a learning rate $\alpha = 0.1$, and $\gamma \approx 0.9615$.

19.5 Compare Almgren-Chriss (Chapter 13, Section 13.3.5) and a reinforcement-learning execution agent (Section 19.4.1) along three dimensions: the data required to use each, the interpretability of the resulting trading schedule, and each approach's ability to adapt mid-execution to an order book that behaves differently than assumed.

19.6 Describe a reward function for a portfolio-rebalancing agent (Section 19.4.3) that would encourage the reward hacking Section 19.6 warns about, and propose a specific, concrete change to that reward function that would discourage it.

19.7 Explain why off-policy evaluation (Section 19.5.2) becomes less reliable the more a newly trained policy's behavior diverges from the policy that generated the available historical data, and describe one practical consequence this has for how a bank should stage the rollout of a new trading agent.

19.8 A market-making desk considers replacing its Avellaneda-Stoikov quoting model (Chapter 13, Section 13.4) with a learned reinforcement-learning policy. List three pieces of evidence a model validator

Wulin.Suo@Queensu.ca

should require before approving this replacement, drawing on both this chapter's discussion and Chapter 8's model-risk-management framework.

19.9 Explain why maximizing expected log-wealth (the Kelly criterion, Section 19.4.3) produces a different, generally more conservative allocation than maximizing expected wealth directly, and describe why this makes the Kelly fraction a useful sanity check for a reinforcement-learning portfolio agent trained on a simplified simulated market.

19.10 Using Section 19.4.6's discussion, explain why a reinforcement-learning execution agent trained against a fixed historical simulator might perform worse than expected once deployed at large enough scale to influence the very order flow it was trained on, and describe one practical step a firm could take to detect this problem before it causes significant losses.

19.11 Explain why behavioral cloning (Section 19.3.4) can fail badly in a state the expert trader rarely visited, even though it performs well on states similar to the expert's typical behavior, and describe how inverse reinforcement learning's approach, inferring a reward function rather than directly imitating actions, is intended to address this specific weakness.

19.12 Section 19.2.4's desk has sent four orders to venue A, giving $Q(A) = 1.60$, and six to venue B, giving $Q(B) = 1.85$. (a) A fifth order to A fills at 3.00 basis points; apply the running-average rule to compute the updated $Q(A)$, and state which venue the greedy policy now selects. (b) A single fill has a standard deviation of 2.0 basis points. Compute the standard error of each of the two estimates and explain why the switch in part (a) is far weaker evidence than it looks. (c) A fourth venue D, whose true mean of 2.4 basis points beats every existing venue, is now connected, and the desk keeps $\varepsilon = 0.2$ with exploration spread uniformly over all four. Compute the new permanent shortfall $\varepsilon(\mu^* - \bar{\mu})$, compare it to the three-venue figure, and explain why adding a venue better than all the others nonetheless raises the per-order cost of a fixed exploration rate.

19.13 After 200 orders, Section 19.2.4's desk has sent $N = (12, 180, 8)$ orders to venues A, B, C, with estimates $Q = (1.40, 1.95, 0.60)$ basis points; the fill noise is $\sigma = 2.0$. (a) Using equation (19.3) with $t = 201$, compute each venue's upper confidence bound and state where UCB routes order 201. (b) Write down each venue's Thompson-sampling posterior (flat prior, known σ), and compute the probability that venue A's draw exceeds venue B's, and likewise for C. (c) UCB and Thompson sampling disagree about order 201 in a characteristic way. Describe the disagreement and explain why both rules still route most of the next thousand orders to B.

19.14 Section 19.4.5's simulator draws price paths under the real-world drift μ. (a) Using the companion notebook's closed form for $e^{-rT}\mathbb{E}[(S_T - K)^+]$ under drift μ, compute this quantity for the \$50-strike call at $\mu = 4\%$, 8% and 12%, and identify which value is the arbitrage-free price. (b) Explain why the hedging-risk figures in Table 19.5 are correctly computed under the real-world drift while a price is not. (c) A colleague proposes training a reinforcement-learning agent to quote option prices by rewarding it with discounted payoffs on historical price paths. State what number it will converge to and propose one fix.

19.15 **Challenge (real data).** Download at least three years of daily prices for a real, liquid stock (via `yfinance`). (a) Discretize the state space into a small number of buckets based on the sign and magnitude of the trailing five-day return, in the spirit of Section 19.3.1's three-state example, and train a tabular Q-learning agent with actions {buy, hold, sell} and a reward equal to the next day's realized return (long) or its negative (short), using this chapter's Q-learning update rule. (b) Compare the trained agent's cumulative return over the full sample to a simple buy-and-hold benchmark, then re-run training on only the first half of the sample and evaluate the resulting policy on the second half, and report whether the agent's apparent advantage survives this split. (c) Using Section 19.5.2's discussion of off-policy evaluation, explain why backtesting a single historical price path is an especially weak form of policy evaluation for a trading agent, and describe one practical safeguard, drawing on Section 19.4.6 or Chapter 8's model-risk-management framework, a firm should require before deploying such an agent with real capital.

Wulin.Suo@Queensu.ca

19.9 Further Reading

- Sutton and Barto (2018), *Reinforcement Learning*. The standard textbook treatment of the Markov decision process framework, the Bellman equation, and value-based and policy-based methods developed throughout this chapter.
- Lattimore and Szepesvári (2020), *Bandit Algorithms*. A modern, rigorous treatment of the bandit problem, including the regret bounds behind Section 19.2.4's comparison of ε-greedy, UCB, and Thompson sampling, and contextual bandits.
- Mnih et al. (2015), "Human-Level Control through Deep Reinforcement Learning." The original Deep Q-Network paper underlying Section 19.3.2's function-approximation extension of tabular Q-learning.
- Schulman et al. (2017), *Proximal Policy Optimization Algorithms*. The algorithm underlying the modern policy-gradient methods discussed in Section 19.3.3.
- Nevmyvaka et al. (2006), "Reinforcement Learning for Optimized Trade Execution." An early and influential application of tabular Q-learning to real limit-order-book execution data, underlying Section 19.4.1's discussion.
- Almgren and Chriss (2001), "Optimal Execution of Portfolio Transactions." The closed-form optimal-execution model this chapter's Section 19.2.2 contrasts with a learned, model-free alternative.
- Avellaneda and Stoikov (2008), "High-Frequency Trading in a Limit Order Book." The closed-form inventory-based market-making model Section 19.4.1 generalizes into a learned quoting policy.
- Buehler et al. (2019), "Deep Hedging." The paper that introduced Section 19.4.5's risk-measure-based, neural-network approach to hedging under transaction costs and other frictions.
- Abbeel and Ng (2004), "Apprenticeship Learning via Inverse Reinforcement Learning." The foundational paper on inverse reinforcement learning discussed in Section 19.3.4.

Wulin.Suo@Queensu.ca